

Reviewer Pack — Decision Tree Depth and Communication Complexity via Lifting

Entry eec3f948dcf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 00:11:15 UTC

Decision Tree Depth and Communication Complexity via Lifting

Entry ID: eec3f948dcf **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 00:11:15 UTC

1. Conjecture

Field A (mathematical branch): DECISION_TREE_DEPTH **Field B** (complexity object): COMMUNICATION_COMPLEXITY

Statement:

For any Boolean function f on n variables, if the decision tree depth of f is d , then the deterministic communication complexity $R_{\{1/3\}}(f)$ is $\Omega(d / \log n)$.

Rationale (proposer's reasoning):

This conjecture explores the relationship between the depth of a decision tree and the communication complexity of the function, leveraging lifting techniques to establish a lower bound. If true, it would imply that functions with deeper decision trees require more communication, which is a key insight in understanding the interplay between these complexity measures.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 98c49723d2fe2600

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n in $\{8, 16, 24, 32, 40\}$ and depths $d \in \{1..n\}$, test ≥ 20 Boolean functions per (n, d) cell (≥ 5 seeds $\times$ structured families incl. parity, AND-OR trees, addressing) computing deterministic CC $D(f)$ and decision-tree depth $DT(f)$. Conjecture is SUPPORTED iff $D(f) \geq c \cdot DT(f) / \log_2(n)$ holds in $\geq 90\%$ of (n, d, seed) trials for some fitted $c > 0$, with mean ratio $D(f) \cdot \log_2(n) / DT(f) \geq 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - decision tree depth communication complexity lifting theorem - query complexity communication complexity lifting gadget deterministic - deterministic communication complexity decision tree depth lower bound Boolean function

Top relevant hits considered: - [http://arxiv.org/abs/2512.11833v1] Soft Decision Tree classifier: explainable and extendable PyTorch implementation - [http://arxiv.org/abs/1302.4207v1] A composition theorem for decision tree complexity - [http://arxiv.org/abs/2105.01963v3] One-way communication complexity and non-adaptive decision trees - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/nlin/0101006v1] On Complexity and Emergence - [http://arxiv.org/abs/1512.01210v1] Nearly optimal separations between communication (or query) complexity and partitions - [http://arxiv.org/abs/1810.08668v2] Parity Decision Tree Complexity is Greater Than Granularity - [http://arxiv.org/abs/0908.4453v1] Depth-Independent Lower bounds on the Communication Complexity of Read-Once Boolean Formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def decision_tree_depth(f, n):
        if len(f) == 2**n:
            return 1
        mid = 2**(n-1)
        left = f[:mid]
        right = f[mid:]
        return 1 + max(decision_tree_depth(left, n-1), decision_tree_depth(right, n-1))

    def deterministic_communication_complexity(f, n):
        if len(f) == 2**n:
            return 0
        mid = 2**(n-1)
        left = f[:mid]
        right = f[mid:]
        return max(deterministic_communication_complexity(left, n-1), deterministic_communication_
```

```

n_values = [8, 16, 24, 32, 40]
depths = range(1, 5)
instances_tested = 0
total_ratio = 0.0
support_count = 0

for n in n_values:
    for d in depths:
        for _ in range(20):
            f = generate_boolean_function(n)
            dt_depth = decision_tree_depth(f, n)
            cc = deterministic_communication_complexity(f, n)
            instances_tested += 1
            ratio = cc * math.log2(n) / dt_depth
            total_ratio += ratio
            if dt_depth >= d and ratio >= 0.5:
                support_count += 1

mean_ratio = total_ratio / instances_tested
conjecture_holds = support_count / (len(n_values) * len(depths) * 20) >= 0.9
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "communication_complexity_ratio",
    "metric_value": mean_ratio,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s (returncode 124) without producing any RESULT line or data, so the pre-registered support condition cannot be evaluated. | next: Reduce the parameter grid (e.g., restrict to $n \in \{8,16,24\}$ and sample fewer depths) and/or optimize the deterministic CC computation to fit within the time budget.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	780552
2	preregistration	claude_max	opus	0	0	8539
3	novelty	claude_max	opus	0	0	2667
4	novelty	claude_max	opus	0	0	8600
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11058
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9253
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8868
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9873
9	judge	claude_max	opus	0	0	4199

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 843609 ms total latency. Provider mix: {'ollama_remote': 5, 'claude_max': 4}

(full prompt+response transcripts available in `research/audit/eeec3f948dcf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eeec3f948dcf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/eeec3f948dcf.tar.gz` (if generated)